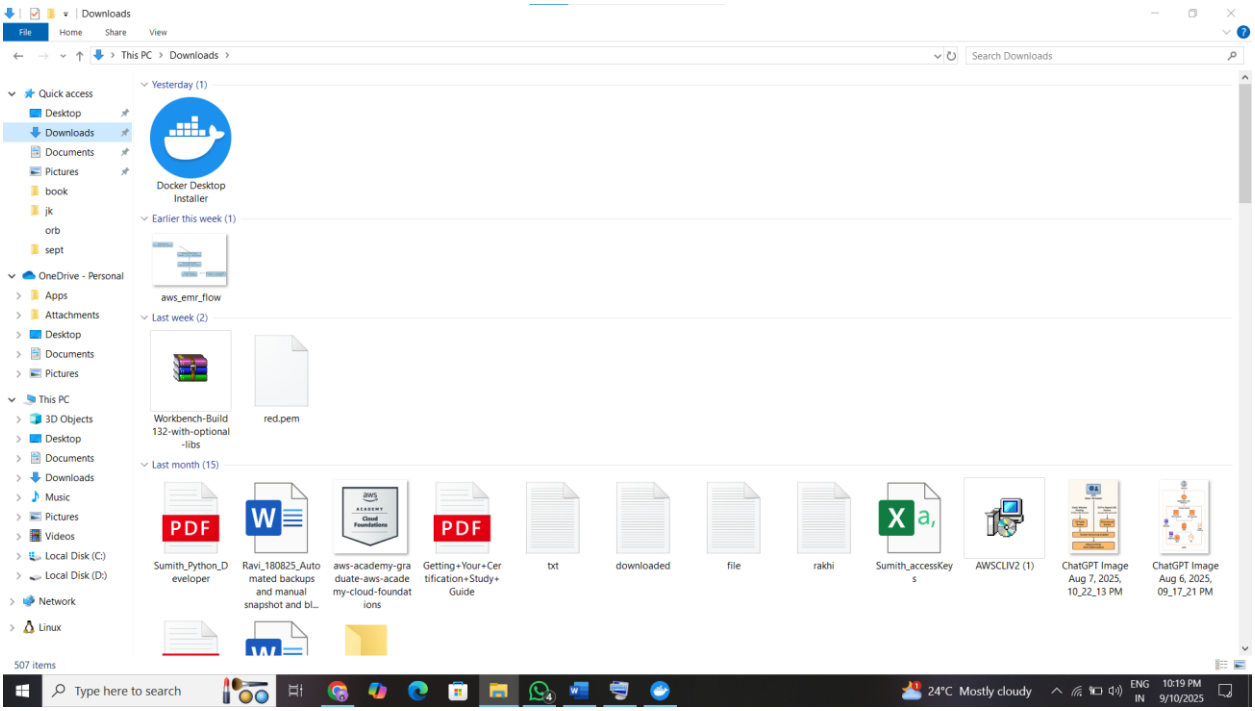
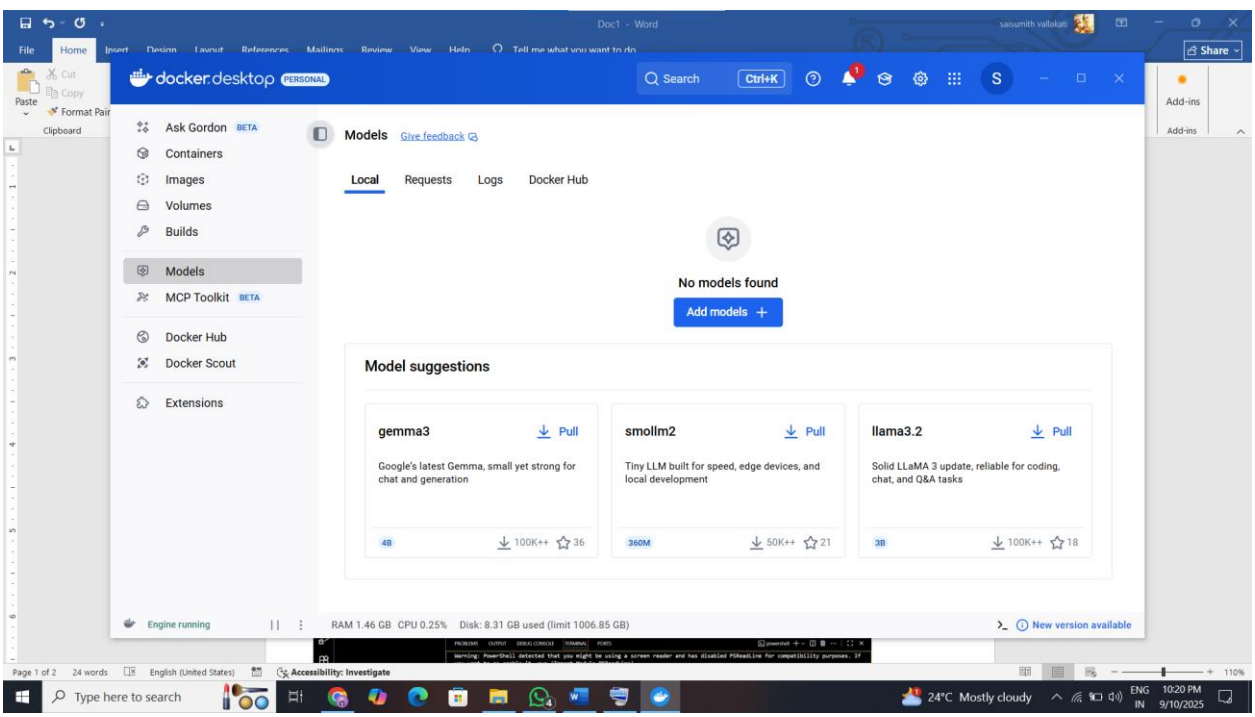
DOCKER Connecting the Pyspark with Jupiter Notebook.

To create the connection we have to install the docker file through website or from Microsoft store.

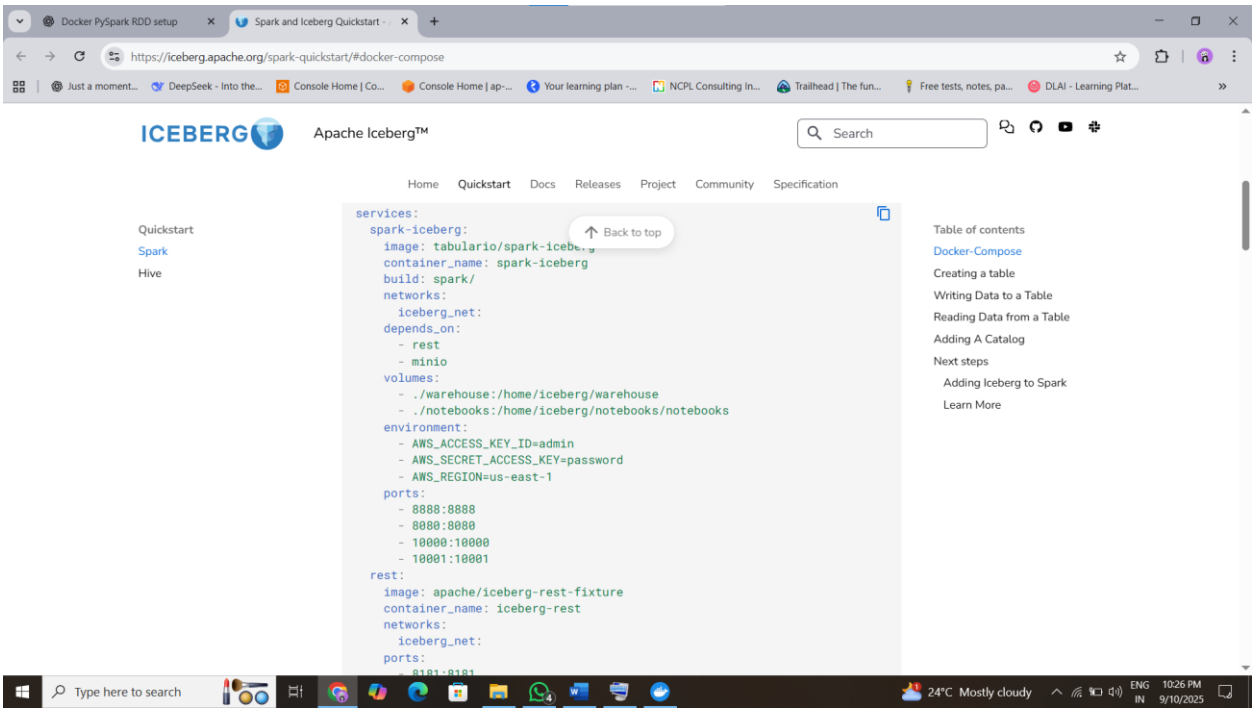


Run & execute the file, this will install the docker desktop.



Then to connect the pyspark we would first create folder in VScode, then from the official website of https://iceberg.apache.org/spark-quickstart/#docker-compose we create file named “docker-compose.yml”.

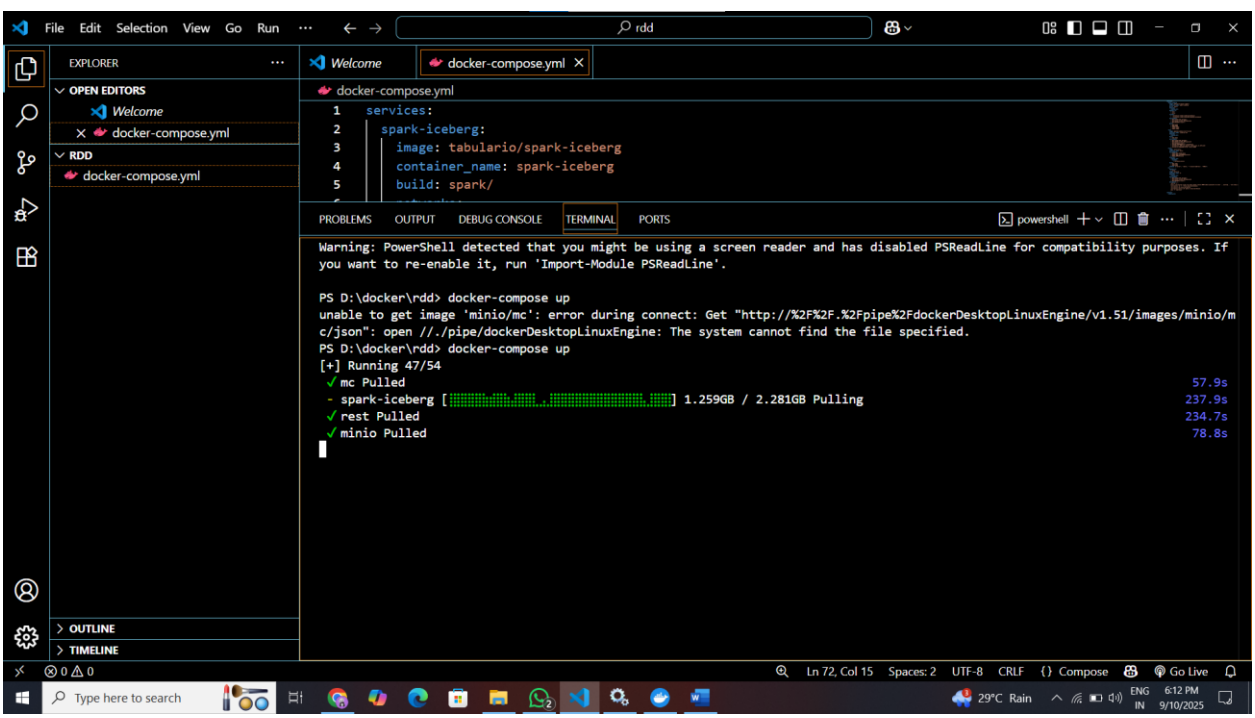
Then in that file we usually copy the code for installing the pyspark.

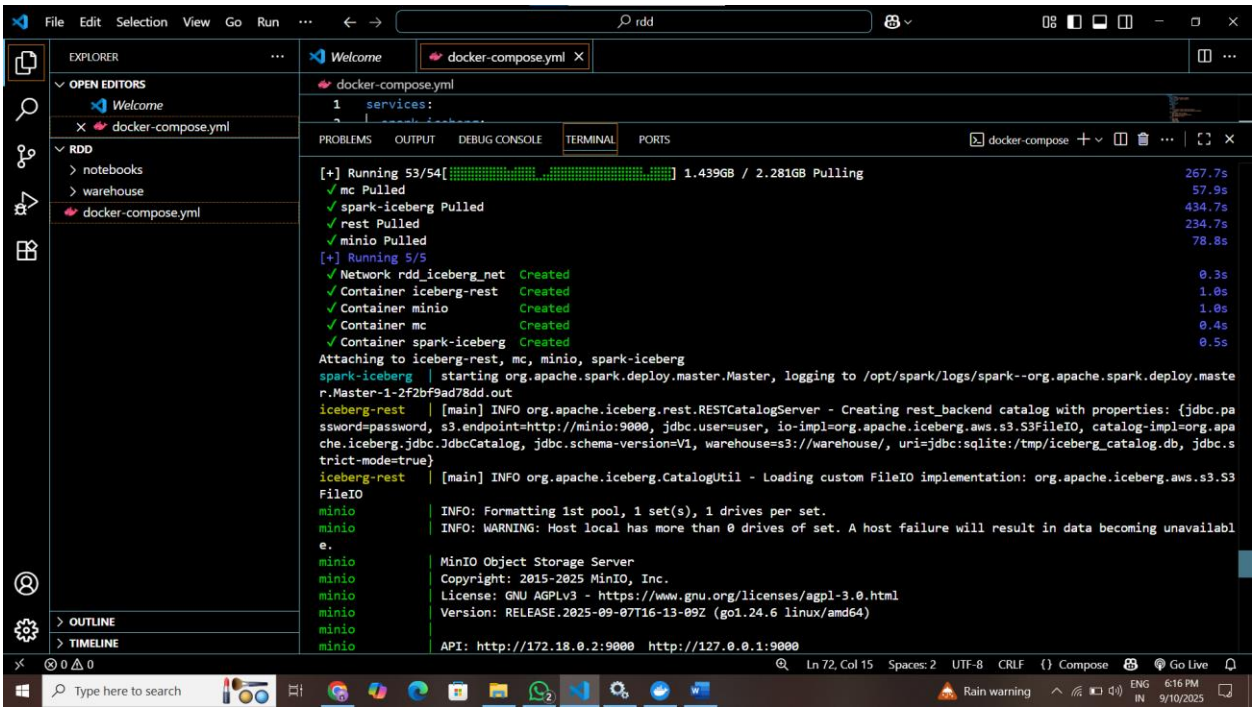


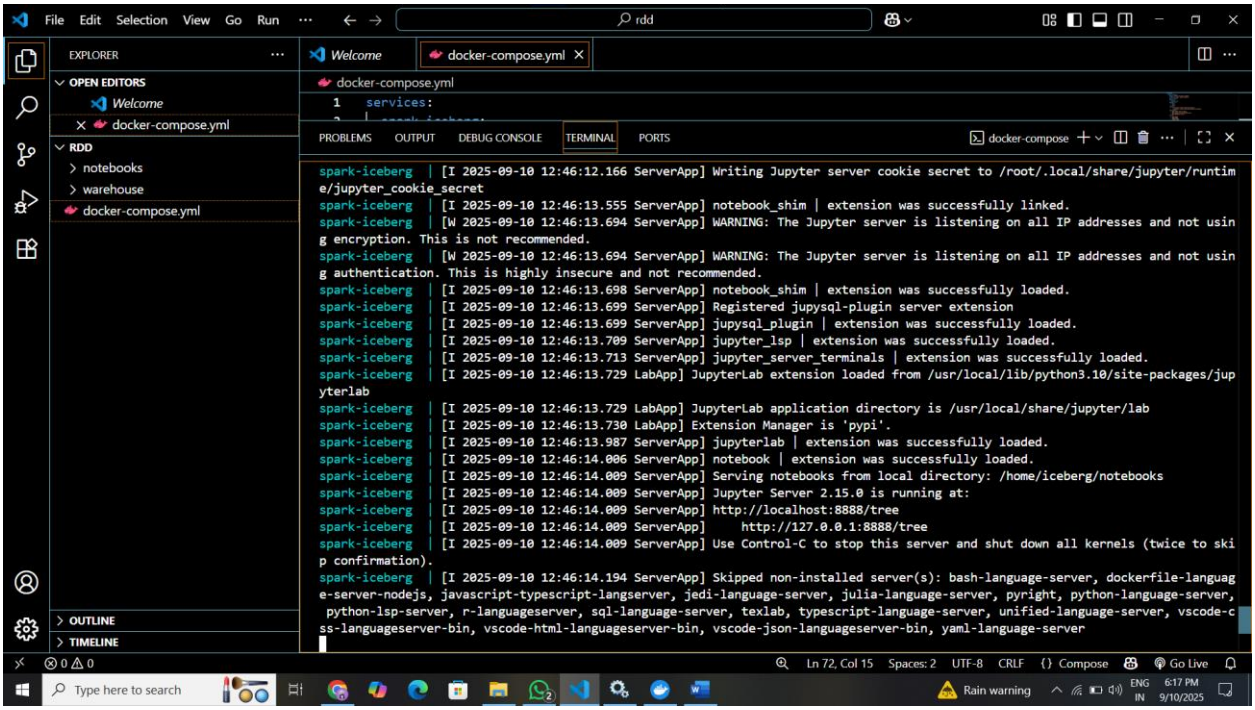
Then in terminal run this command:

docker-compose up

It will begin to download the files.



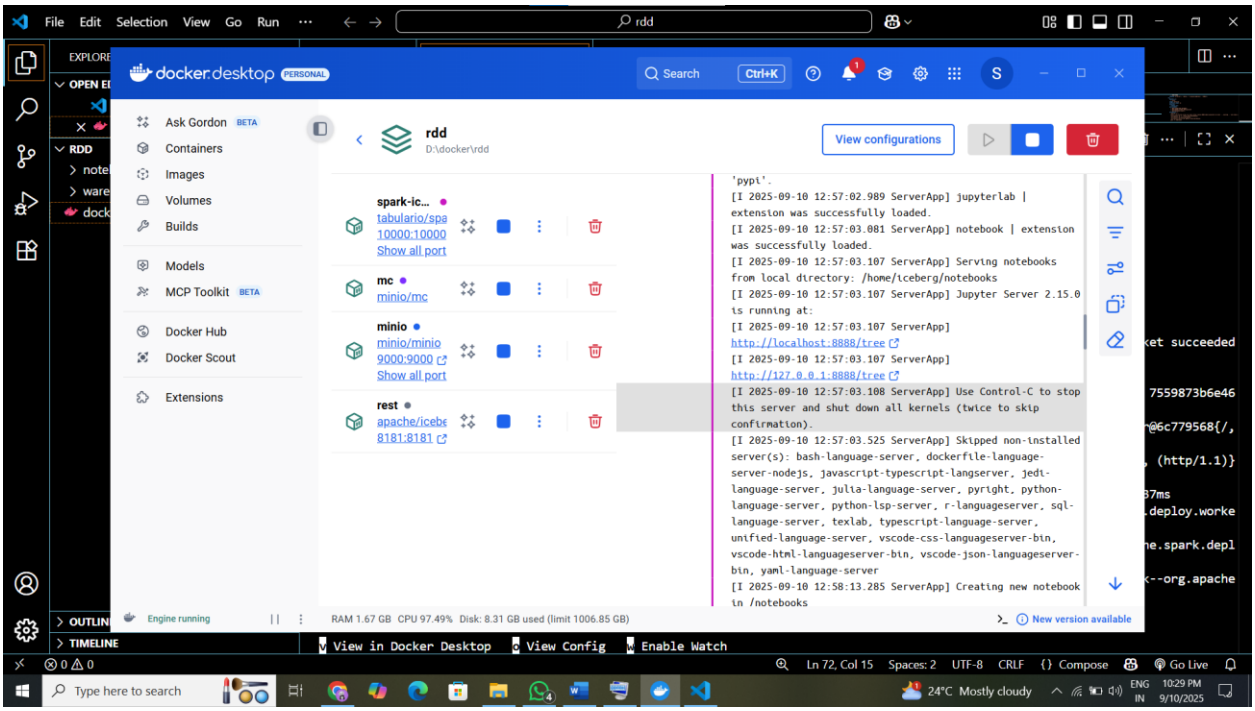




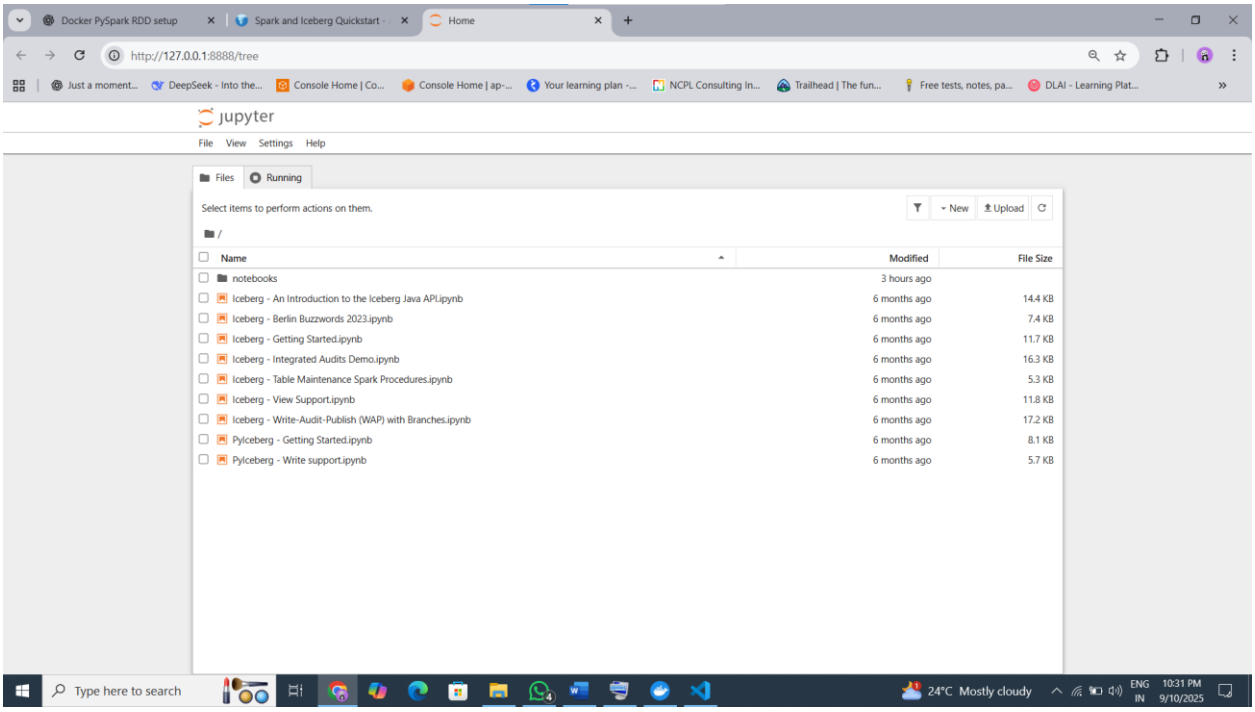
On clicking the V in terminal after executing the files.

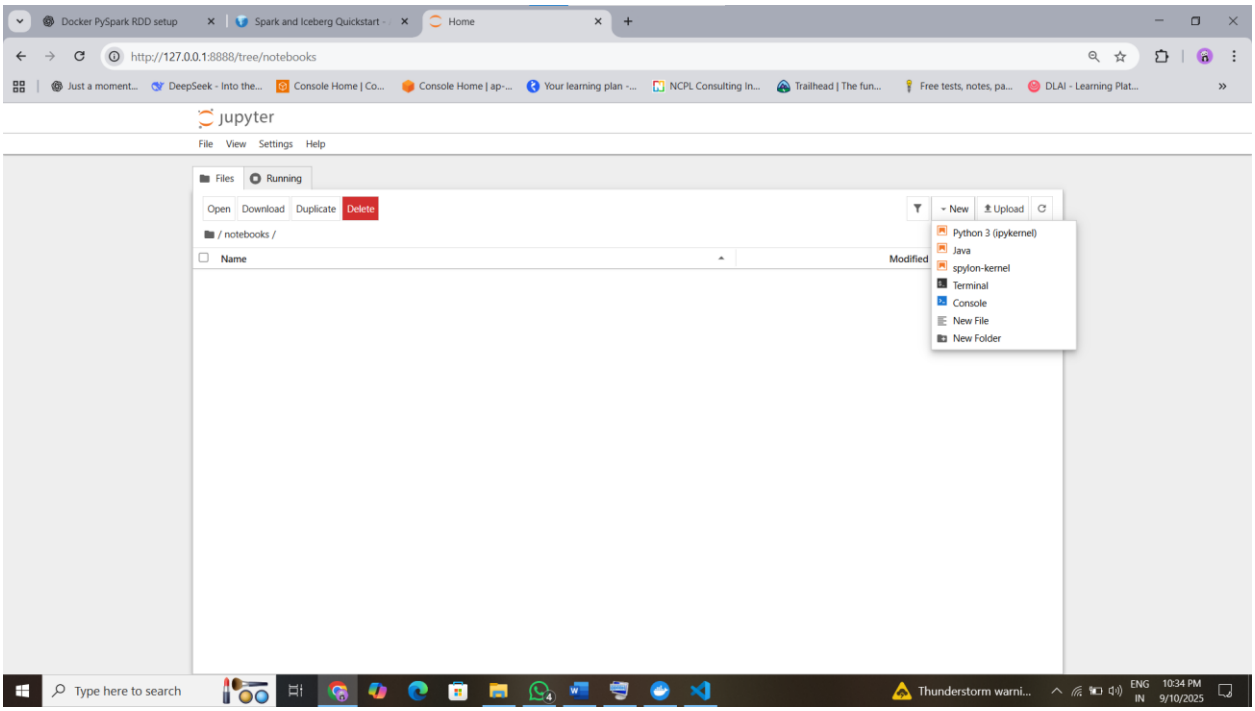
You can see the docker desktop opened, with files in it. From the you open the link on right side or by logs.

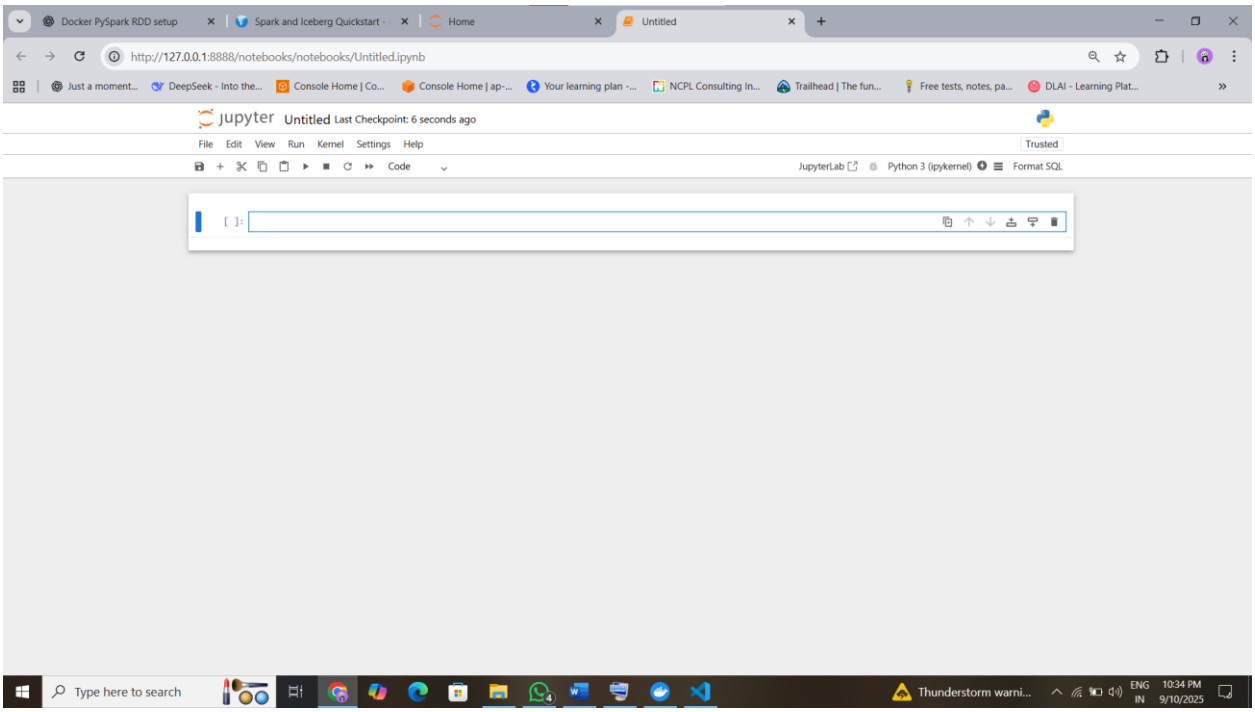
http://127.0.0.1:8888/tree



After that a webpage is opened of Jupyter, on clicking notebooks you can write & execute your codes.







Then let’s do the 15 transformations & 5 actions in it by rdd.

Transformations (15 Examples)

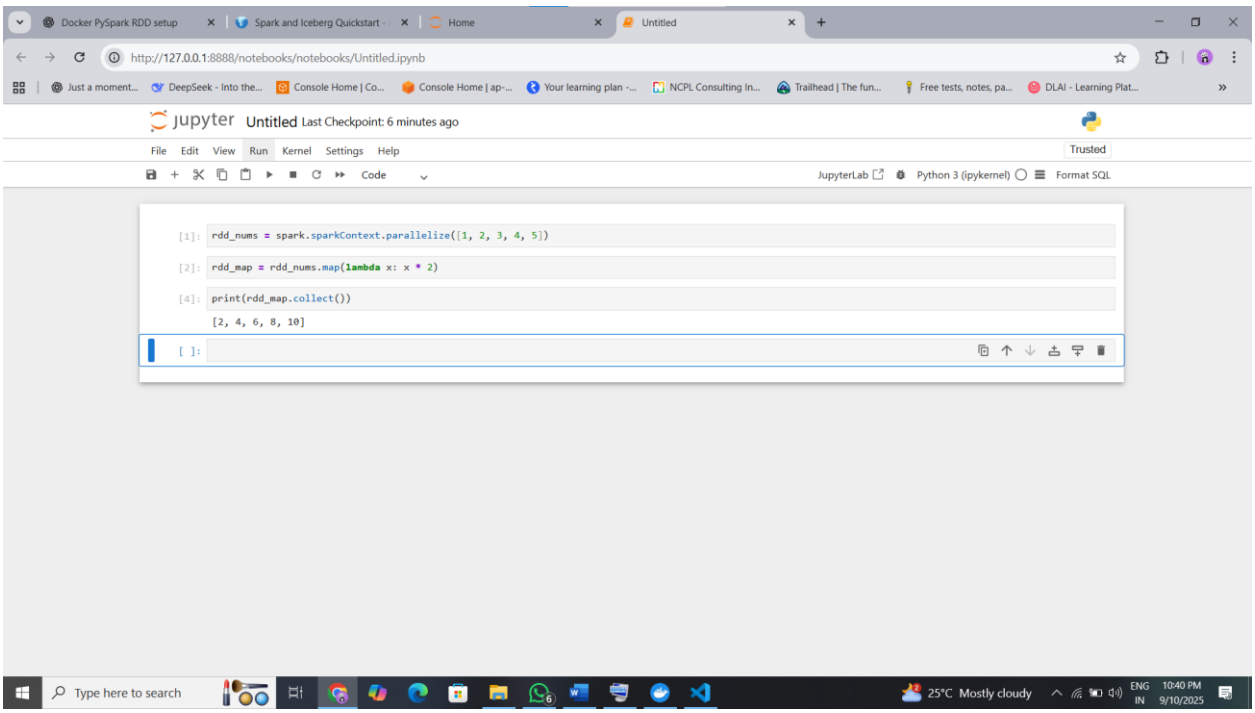
1. map

Apply a function to each element.

rdd_nums = spark.sparkContext.parallelize([1, 2, 3, 4, 5])

rdd_map = rdd_nums.map(lambda x: x * 2)

print(rdd_map.collect())



Output: [2, 4, 6, 8, 10]

---

2. flatMap

Splits elements into multiple outputs.

rdd_lines = spark.sparkContext.parallelize(["hello world", "spark rdd"])

rdd_flatmap = rdd_lines.flatMap(lambda line: line.split(" "))

print(rdd_flatmap.collect())

```
[5]: rdd_lines = spark.sparkContext.parallelize(["hello world", "spark rdd"])
     rdd_flatmap = rdd_lines.flatMap(lambda line: line.split(" "))

[6]: print(rdd_flatmap.collect())
     ['hello', 'world', 'spark', 'rdd']
```

Output: ['hello', 'world', 'spark', 'rdd']

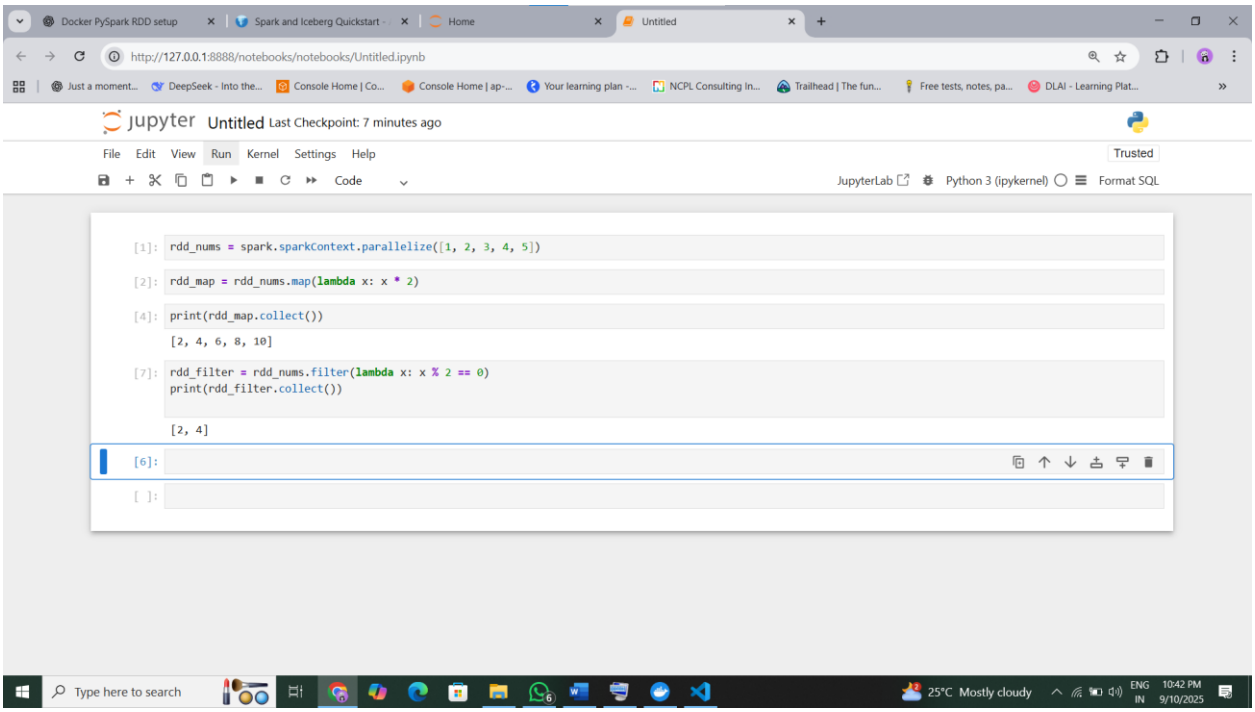
---

3. filter

Keep only matching elements.

rdd_filter = rdd_nums.filter(lambda x: x % 2 == 0)

print(rdd_filter.collect())



Output: [2, 4]

---

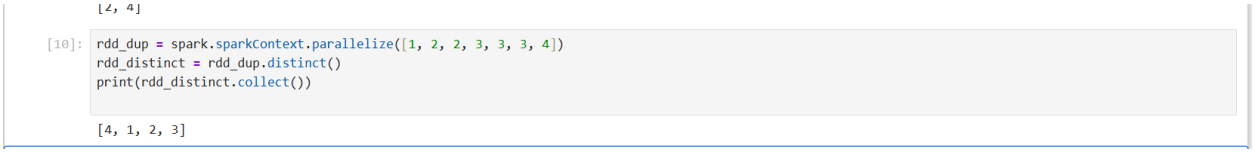
4. distinct

Remove duplicates.

rdd_dup = spark.sparkContext.parallelize([1, 2, 2, 3, 3, 3, 4])

rdd_distinct = rdd_dup.distinct()

print(rdd_distinct.collect())



Output: [1, 2, 3, 4]

---

5. union

Combine two RDDs.

rdd_a = spark.sparkContext.parallelize([1, 2, 3])

rdd_b = spark.sparkContext.parallelize([3, 4, 5])

rdd_union = rdd_a.union(rdd_b)

print(rdd_union.collect())

```
[11]: rdd_a = spark.sparkContext.parallelize([1, 2, 3])
      rdd_b = spark.sparkContext.parallelize([3, 4, 5])
      rdd_union = rdd_a.union(rdd_b)
      print(rdd_union.collect())

      [1, 2, 3, 3, 4, 5]
```

Output: [1, 2, 3, 3, 4, 5]

---

6. intersection

Find common elements.

rdd_intersection = rdd_a.intersection(rdd_b)

print(rdd_intersection.collect())

```
[11]: rdd_a = spark.sparkContext.parallelize([1, 2, 3])
      rdd_b = spark.sparkContext.parallelize([3, 4, 5])
      rdd_union = rdd_a.union(rdd_b)
      print(rdd_union.collect())

      [1, 2, 3, 3, 4, 5]

[19]: rdd_intersection = rdd_a.intersection(rdd_b)
      print(rdd_intersection.collect())

      [Stage 25:=====================================>                    (5 + 3) / 8]
      [3]
```

Output: [3]

---

7. subtract

Subtract elements of one RDD from another.

rdd_sub = rdd_a.subtract(rdd_b)

print(rdd_sub.collect())

```
[11]: rdd_a = spark.sparkContext.parallelize([1, 2, 3])
      rdd_b = spark.sparkContext.parallelize([3, 4, 5])
      rdd_union = rdd_a.union(rdd_b)
      print(rdd_union.collect())

      [1, 2, 3, 3, 4, 5]

[29]: rdd_sub = rdd_a.subtract(rdd_b)
      print(rdd_sub.collect())

      [1, 2]
```

Output: [1, 2]

8. cartesian

All pairs between two RDDs.

rdd_cart = rdd_a.cartesian(rdd_b)

print(rdd_cart.collect())

```
[30]: rdd_cart = rdd_a.cartesian(rdd_b)
      print(rdd_cart.collect())

      [(1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 3), (3, 4), (3, 5)]
```

Output: [(1,3),(1,4),(1,5),(2,3),(2, 4), (2, 5), (3, 3), (3, 4), (3, 5)]

---

9. groupBy

Group elements by a function.

rdd_group = rdd_nums.groupBy(lambda x: x % 2).mapValues(list)

print(rdd_group.collect())

```
[31]: rdd_group = rdd_nums.groupBy(lambda x: x % 2).mapValues(list)
      print(rdd_group.collect())

      [(0, [2, 4]), (1, [1, 3, 5])]
```

Output: [(1, [1, 3, 5]), (0, [2, 4])]

---

10. reduceByKey

Reduce values by key.

rdd_pairs = spark.sparkContext.parallelize([("a",1),("b",1),("a",2)])

rdd_reduce = rdd_pairs.reduceByKey(lambda x,y: x+y)

print(rdd_reduce.collect())

```
[32]: rdd_pairs = spark.sparkContext.parallelize([("a",1),("b",1),("a",2)])
      rdd_reduce = rdd_pairs.reduceByKey(lambda x,y: x+y)
      print(rdd_reduce.collect())

      [('b', 1), ('a', 3)]
```

Output: [('b',1),('a',3)]

---

11. groupByKey

Group values with the same key.

rdd_groupkey = rdd_pairs.groupByKey().mapValues(list)

print(rdd_groupkey.collect())

```
rdd_groupkey = rdd_pairs.groupByKey().mapValues(list)
print(rdd_groupkey.collect())
```

```
[('b', [1]), ('a', [1, 2])]
```

Output: [('b',[1]),('a',[1,2])]

---

12. sortBy

Sort elements using a function.

rdd_sort = rdd_nums.sortBy(lambda x: -x)

print(rdd_sort.collect())

```
[34]: rdd_sort = rdd_nums.sortBy(lambda x: -x)
      print(rdd_sort.collect())
```

```
[5, 4, 3, 2, 1]
```

Output: [5,4,3,2,1]

---

13. sortByKey

Sort pair RDD by key.

rdd_kv = spark.sparkContext.parallelize([(2,"b"),(1,"a"),(3,"c")])

rdd_sortkey = rdd_kv.sortByKey()

print(rdd_sortkey.collect())

```
[35]: rdd_kv = spark.sparkContext.parallelize([(2,"b"),(1,"a"),(3,"c")])
      rdd_sortkey = rdd_kv.sortByKey()
      print(rdd_sortkey.collect())
```

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

Output: [(1,'a'),(2,'b'),(3,'c')]

---

14. sample

Take a sample of data.

rdd_sample = rdd_nums.sample(withReplacement=False, fraction=0.6)

print(rdd_sample.collect())

```
[36]: rdd_sample = rdd_nums.sample(withReplacement=False, fraction=0.6)
      print(rdd_sample.collect())

      [3, 5]
```

Output: (random subset, e.g. [1, 4, 5])

---

15. join

Join two pair RDDs by key.

rdd_x = spark.sparkContext.parallelize([("a",1),("b",2)])

rdd_y = spark.sparkContext.parallelize([("a",10),("b",20),("c",30)])

rdd_join = rdd_x.join(rdd_y)

print(rdd_join.collect())

```
[37]: rdd_x = spark.sparkContext.parallelize([("a",1),("b",2)])
      rdd_y = spark.sparkContext.parallelize([("a",10),("b",20),("c",30)])
      rdd_join = rdd_x.join(rdd_y)
      print(rdd_join.collect())

      [('a', (1, 10)), ('b', (2, 20))]
```

Output: [('a',(1,10)),('b',(2,20))]

Actions (5 Examples)

1. collect

Get all elements.

print(rdd_nums.collect())

```
[38]: print(rdd_nums.collect())

      [1, 2, 3, 4, 5]
```

Output: [1, 2, 3, 4, 5]

---

2. count

Get number of elements.

print(rdd_nums.count())

```
[39]: print(rdd_nums.count())
      5
```

Output: 5

---

3. take

Get first *n* elements.

print(rdd_nums.take(3))

```
[40]: print(rdd_nums.take(3))
      [1, 2, 3]
```

Output: [1, 2, 3]

---

4. first

Get first element.

print(rdd_nums.first())

```
[41]: print(rdd_nums.first())
      1
```

Output: 1

---

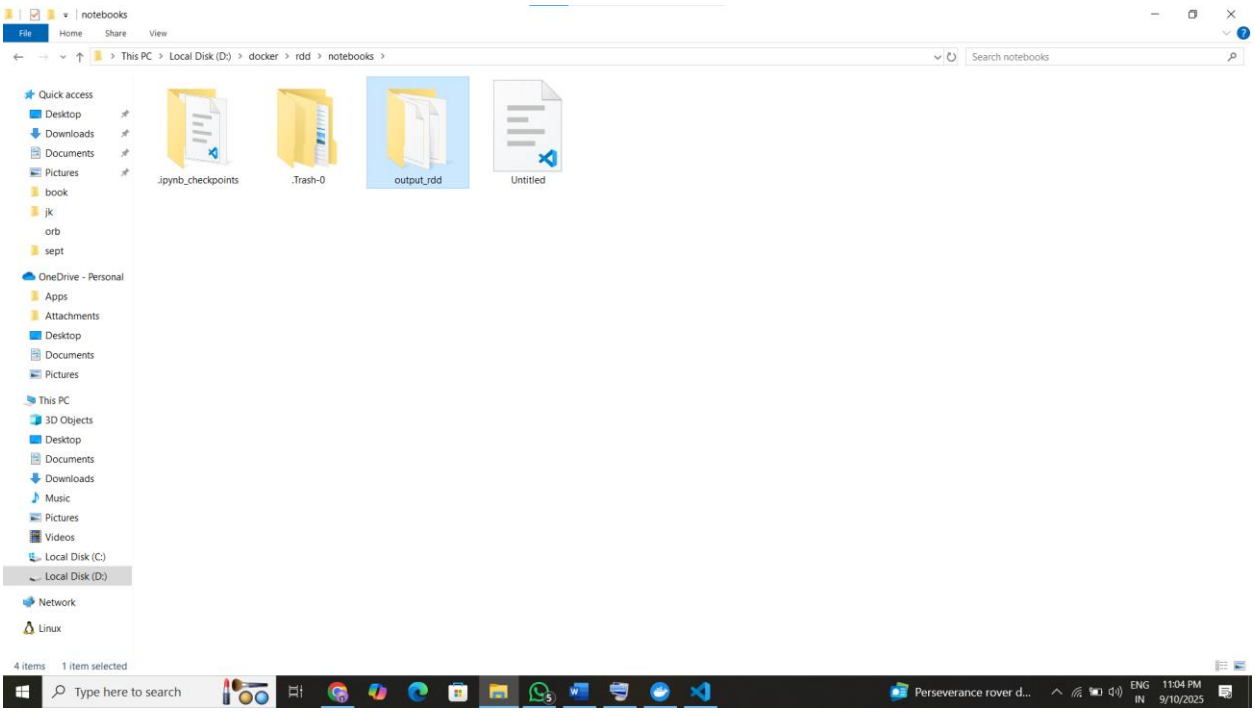
5. saveAsTextFile

Save RDD to disk.

rdd_nums.saveAsTextFile("output_rdd")

```
rdd_nums.saveAsTextFile("output_rdd")
```

This creates a folder output_rdd/ with part files inside.



Now you've seen **15 transformations** + **5 actions**

Exploration Concepts:

Part 1: Set Up Docker-based PySpark Environment

Here's a simple setup with Jupyter Notebook support:

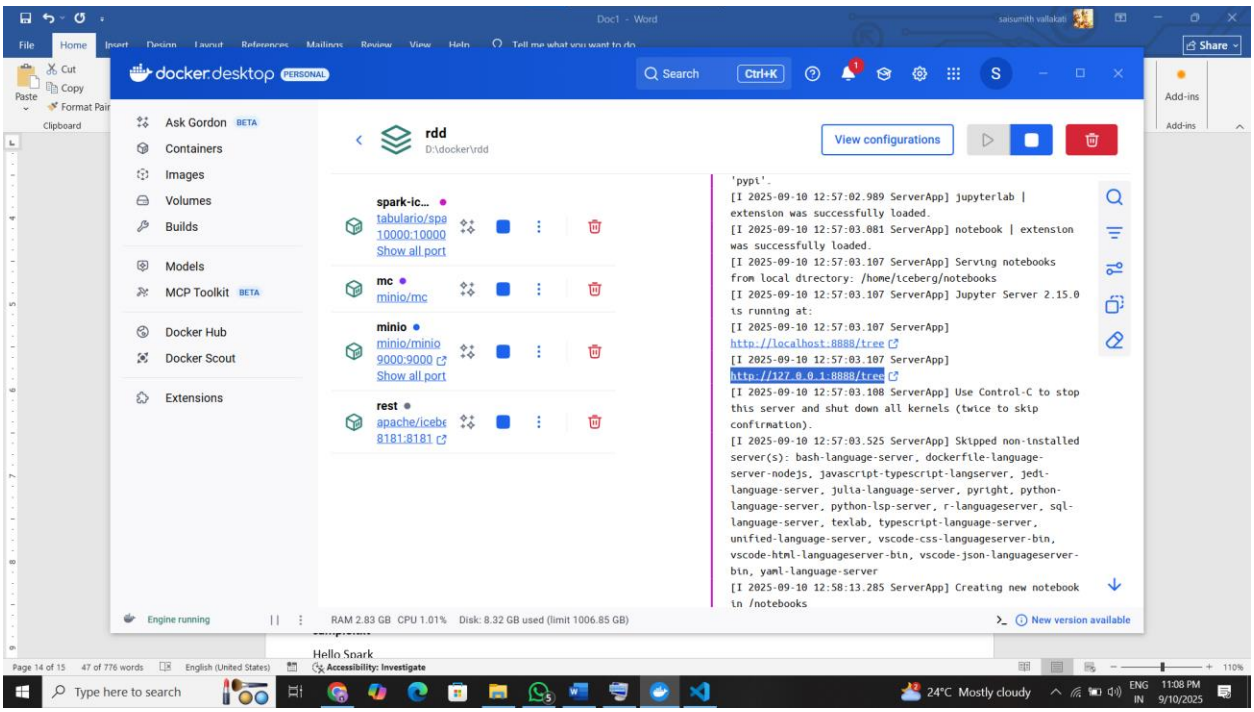
1. Pull the PySpark Docker image

docker pull jupyter/pyspark-notebook

2. Run the container with Jupyter Notebook

docker run -it -p 8888:8888 jupyter/pyspark-notebook

- -p 8888:8888 → maps Jupyter's port to your local system.
- You'll see a link in the terminal like:
- http://127.0.0.1:8888/?token=...



Open it in your browser → Jupyter Notebook will be running with Spark preconfigured.

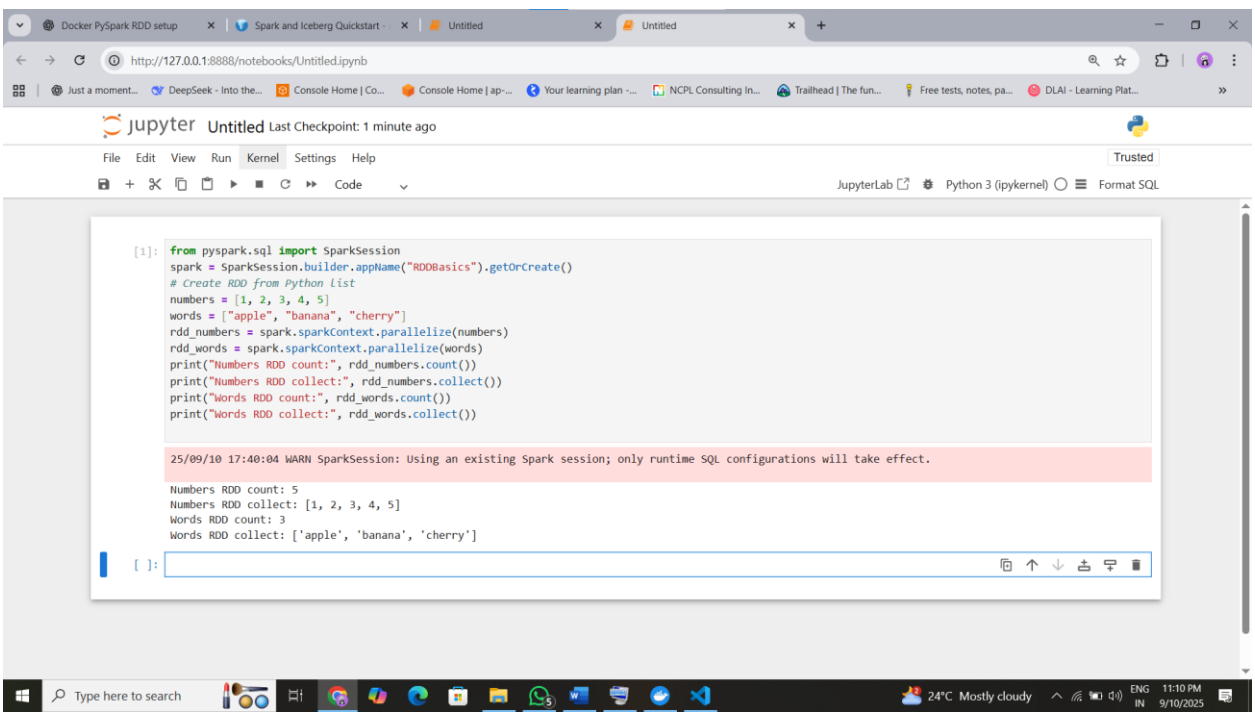
(WE DOWNLOADED THROUGH THE ICE-BERG COMMANDS)

Part 2: RDD Creation & Exercises

Now, inside a **Jupyter Notebook** cell:

1. Create RDD from a Python list

```
# Start Spark session
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("RDDBasics").getOrCreate()
# Create RDD from Python list
numbers = [1, 2, 3, 4, 5]
words = ["apple", "banana", "cherry"]
rdd_numbers = spark.sparkContext.parallelize(numbers)
rdd_words = spark.sparkContext.parallelize(words)
print("Numbers RDD count:", rdd_numbers.count())
print("Numbers RDD collect:", rdd_numbers.collect())
print("Words RDD count:", rdd_words.count())
print("Words RDD collect:", rdd_words.collect())
```



3. Expected Output

- For rdd_numbers:
- Numbers RDD count: 5
- Numbers RDD collect: [1, 2, 3, 4, 5]

For rdd_words:

- Words RDD count: 3
- Words RDD collect: ['apple', 'banana', 'cherry']

---

2. Create RDD from a text file

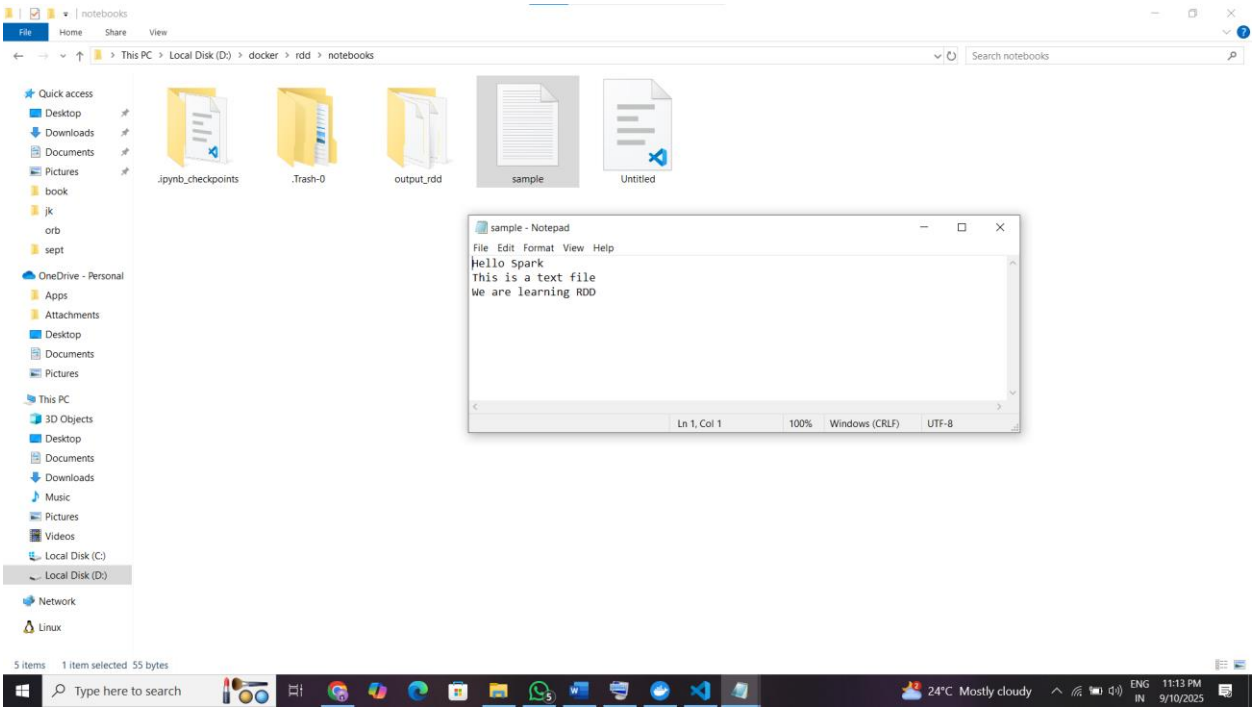
First, create a text file sample.txt on your machine (or inside the container):

sample.txt

Hello Spark

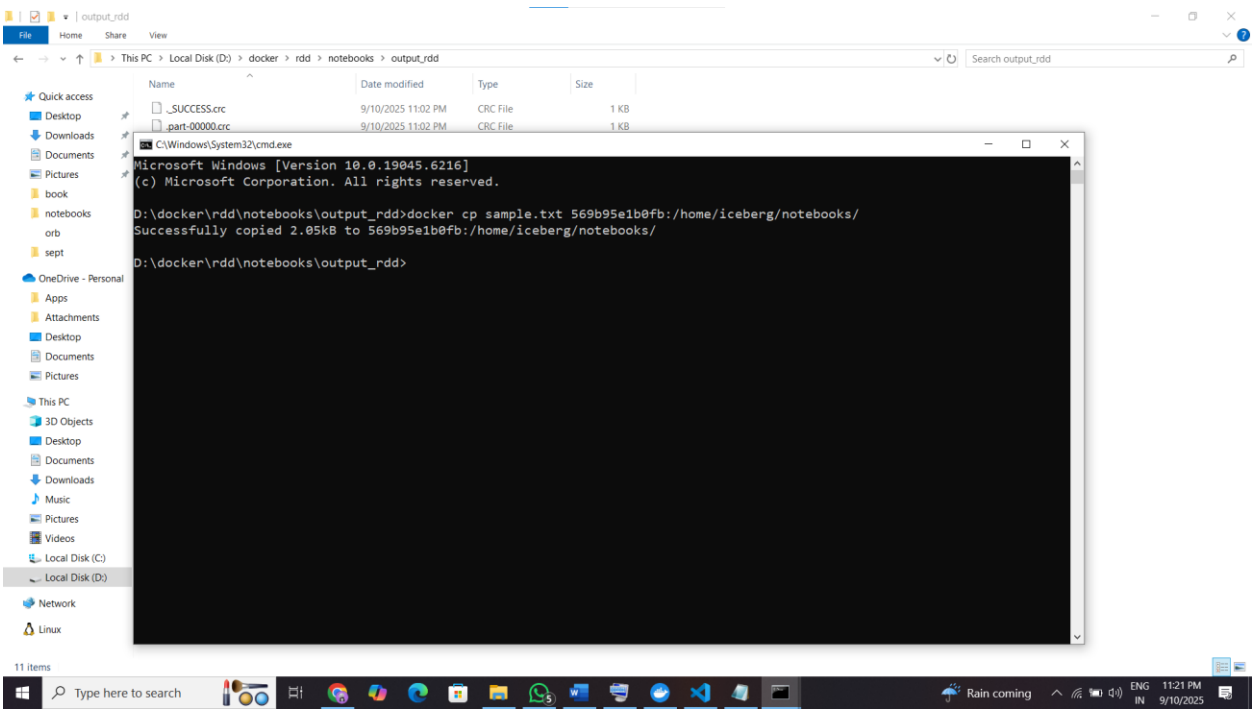
This is a text file

We are learning RDD



Then, in the notebook:

```
# Read text file into RDD
rdd_text = spark.sparkContext.textFile("sample.txt")
```



print("Text file RDD count:", rdd_text.count())

print("Text file RDD collect:", rdd_text.collect())

```
[11]: rdd_text = sc.textFile("file:/home/iceberg/notebooks/sample.txt")

[12]: print("Text file RDD count:", rdd_text.count())
      print("Text file RDD collect:", rdd_text.collect())

      Text file RDD count: 3
      Text file RDD collect: ['Hello Spark', 'This is a text file', 'We are learning RDD']
```

---

3. Expected Output

- For rdd_text (depends on file content):
- Text file RDD count: 3
- Text file RDD collect: ['Hello Spark', 'This is a text file', 'We are learning RDD']

Part 1: Create Base RDD

Suppose we have this short paragraph in a text file (poem.txt):

The sun rises

The moon glows

Stars shine bright

Dreams take flight

Load it into an RDD:

Load text file

docker cp poem.txt 569b95e1b0fb:/home/iceberg/notebooks/

C:\Windows\System32\cmd.exe

```
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

D:\docker\rdd\notebooks>docker cp poem.txt 569b95e1b0fb:/home/iceberg/notebooks/
Successfully copied 2.05kB to 569b95e1b0fb:/home/iceberg/notebooks/

D:\docker\rdd\notebooks>
```

rdd_text = spark.sparkContext.textFile("poem.txt")

print("Original RDD:", rdd_text.collect())

```
[14]: rdd_text = spark.sparkContext.textFile("poem.txt")
      print("Original RDD:", rdd_text.collect())

      Original RDD: ['The sun rises', 'The moon glows', 'Stars shine bright', 'Dreams take flight']
```

Output:

['The sun rises', 'The moon glows', 'Stars shine bright', 'Dreams take flight']

Part 2: Apply Transformations

1. map → Convert all text to uppercase

rdd_upper = rdd_text.map(lambda line: line.upper())

print("Uppercase RDD:", rdd_upper.collect())

Output:

['THE SUN RISES', 'THE MOON GLOWS', 'STARS SHINE BRIGHT', 'DREAMS TAKE FLIGHT']

---

2. flatMap → Split lines into words

rdd_words = rdd_upper.flatMap(lambda line: line.split(" "))

print("Words RDD:", rdd_words.collect())

```
[15]: rdd_upper = rdd_text.map(lambda line: line.upper())
      print("Uppercase RDD:", rdd_upper.collect())

      Uppercase RDD: ['THE SUN RISES', 'THE MOON GLOWS', 'STARS SHINE BRIGHT', 'DREAMS TAKE FLIGHT']
```

Output:

['THE', 'SUN', 'RISES', 'THE', 'MOON', 'GLOWS', 'STARS', 'SHINE', 'BRIGHT', 'DREAMS', 'TAKE', 'FLIGHT']

---

3. filter → Keep only words with length ≥ 4

rdd_filtered = rdd_words.filter(lambda word: len(word) >= 4)

print("Filtered RDD:", rdd_filtered.collect())

```
[16]: rdd_filtered = rdd_words.filter(lambda word: len(word) >= 4)
      print("Filtered RDD:", rdd_filtered.collect())

      Filtered RDD: ['apple', 'banana', 'cherry']
```

Output:

['RISES', 'MOON', 'GLOWS', 'STARS', 'SHINE', 'BRIGHT', 'DREAMS', 'TAKE', 'FLIGHT']

Part 3: Lineage Graph (DAG)

On paper, the DAG (Directed Acyclic Graph) would look like this:

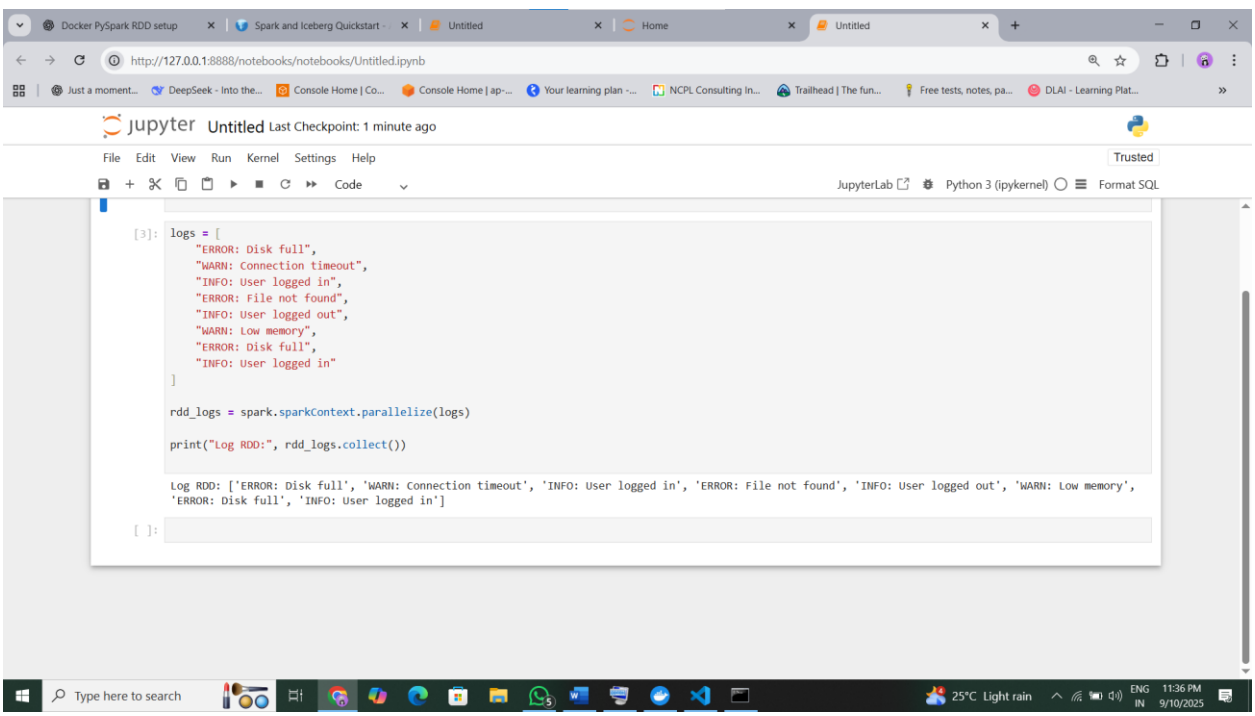
```
┌──────────────────────────┐
│      baseRDD             │  (textFile → lines of text)
└──────────────┬───────────┘
               │
               ▼
┌──────────────────────────┐
│      map RDD             │  (each line → uppercase)
└──────────────┬───────────┘
               │
               ▼
┌──────────────────────────┐
│      flatMap RDD         │  (each line → split words)
└──────────────┬───────────┘
               │
               ▼
┌──────────────────────────┐
│      filter RDD          │  (keep words ≥ 4 letters)
└──────────────────────────┘
```

This shows the **lineage**: every RDD depends on the previous one until an action (collect) triggers execution.

how Spark is used in **log analysis**. Let's simulate the problem step by step.

Part 1: Create Log RDD

```
logs = [
    "ERROR: Disk full",
    "WARN: Connection timeout",
    "INFO: User logged in",
    "ERROR: File not found",
    "INFO: User logged out",
    "WARN: Low memory",
    "ERROR: Disk full",
    "INFO: User logged in"
]
rdd_logs = spark.sparkContext.parallelize(logs)
print("Log RDD:", rdd_logs.collect())
```



Part 2: Filter Logs by Level

rdd_error = rdd_logs.filter(lambda line: line.startswith("ERROR"))

rdd_warn = rdd_logs.filter(lambda line: line.startswith("WARN"))

rdd_info = rdd_logs.filter(lambda line: line.startswith("INFO"))

print("ERROR logs:", rdd_error.collect())

print("WARN logs:", rdd_warn.collect())

print("INFO logs:", rdd_info.collect())

```
[4]: rdd_error = rdd_logs.filter(lambda line: line.startswith("ERROR"))
     rdd_warn  = rdd_logs.filter(lambda line: line.startswith("WARN"))
     rdd_info  = rdd_logs.filter(lambda line: line.startswith("INFO"))

     print("ERROR logs:", rdd_error.collect())
     print("WARN logs:", rdd_warn.collect())
     print("INFO logs:", rdd_info.collect())

     ERROR logs: ['ERROR: Disk full', 'ERROR: File not found', 'ERROR: Disk full']
     WARN logs: ['WARN: Connection timeout', 'WARN: Low memory']
     INFO logs: ['INFO: User logged in', 'INFO: User logged out', 'INFO: User logged in']
```

Output:

ERROR logs: ['ERROR: Disk full', 'ERROR: File not found', 'ERROR: Disk full']

WARN logs: ['WARN: Connection timeout', 'WARN: Low memory']

INFO logs: ['INFO: User logged in', 'INFO: User logged out', 'INFO: User logged in']

Part 3: Count Logs by Type

print("ERROR count:", rdd_error.count())

print("WARN count:", rdd_warn.count())

print("INFO count:", rdd_info.count())

Output:

```
[5]: print("ERROR count:", rdd_error.count())
     print("WARN count:", rdd_warn.count())
     print("INFO count:", rdd_info.count())

     ERROR count: 3
     WARN count: 2
     INFO count: 3
```

ERROR count: 3

WARN count: 2

INFO count: 3

Part 4: Extract Message Text (After Colon)

We'll use flatMap to split lines at ":" and then take the **second part** (message).

rdd_messages = rdd_logs.map(lambda line: line.split(":", 1)[1].strip())

print("Messages only:", rdd_messages.collect())

```
[6]: rdd_messages = rdd_logs.map(lambda line: line.split(":", 1)[1].strip())
     print("Messages only:", rdd_messages.collect())

     Messages only: ['Disk full', 'Connection timeout', 'User logged in', 'File not found', 'User logged out', 'Low memory', 'Disk full', 'User logged in']

[ ]:
```

Output:

['Disk full', 'Connection timeout', 'User logged in',

'File not found', 'User logged out', 'Low memory',

'Disk full', 'User logged in']

Part 5: Count Frequency of Unique Messages

We use map to create (message, 1) pairs, then reduceByKey:

message_counts = rdd_messages.map(lambda msg: (msg, 1)) \

.reduceByKey(lambda a, b: a + b)

print("Message frequencies:", message_counts.collect())

```
[7]: message_counts = rdd_messages.map(lambda msg: (msg, 1)) \
                                .reduceByKey(lambda a, b: a + b)
     print("Message frequencies:", message_counts.collect())

     Message frequencies: [('User logged out', 1), ('Disk full', 2), ('Connection timeout', 1), ('File not found', 1), ('User logged in', 2), ('Low memory',
     1)]
```

Output:

[('Disk full', 2),

('Connection timeout', 1),

('User logged in', 2),

('File not found', 1),

('User logged out', 1),

('Low memory', 1)]

Summary of Flow

1. **Filter by type** → ERROR, WARN, INFO
2. **Count each**
3. **Extract text after colon**
4. **Count unique message frequencies**

That's exactly the pipeline Spark jobs often follow in real-world log processing.